

FinGPT Part 2 开发文档总结

本文档是 `DEVELOPMENT.md` 的中文版本，面向开发者，概括当前仓库的实际实现、主要入口、配置项、数据结构、调试方式，以及继续开发时最需要注意的事项。

仓库是做什么的

这个项目实现了一条本地运行的金融新闻信号流水线：

1. `Agent 1` 从新闻文本中提取结构化指纹 `NewsFingerprint`。
2. `Agent 2` 根据该指纹生成交易信号 `TradingSignal`。
3. `backtest` 模块把同一套流程跑在数据集上，并计算收益与评估指标。

核心思想不是让模型“自己报分数”，而是直接从 vLLM 读取真实 token log-probabilities，再由 Python 做确定性后处理。

整体架构

实时主流程

- `pipeline.py`
 - 实时入口。
 - 从 Alpaca 或 Finnhub 抓新闻。
 - 拼接 `headline + summary` 作为文章输入。
 - 依次调用 `extract_fingerprint()` 和 `generate_signal()`。
 - 结果保存到 `output/signals_<timestamp>.json`。

Agent 1

- `agent1/extractor.py`
 - 加载或复用本地 vLLM 引擎。
 - 用 guided decoding 做结构化事实抽取。
 - 用 logits 方案做情绪分类。
- `agent1/schema.py`
 - 定义 `NewsFingerprint`。
- `agent1/prompt.py`
 - 定义抽取提示词与情绪打分提示词。

Agent 2

- `agent2/reasoner.py`
 - 当 `SHARE_SINGLE_LLM_BETWEEN_AGENTS=true` 时可复用 Agent 1 的 vLLM 引擎。
 - 先生成策略 CoT，再做 A/B/C 打分。
 - 使用 null-context logprobs 做 PMI 校正。
- `agent2/schema.py`
 - 定义 `TradingSignal`。
- `agent2/prompt.py`

- 定义策略提示词、决策前缀和评分 token。

共享 logits 工具

- [vllm_logits_client.py](#)
 - 封装两阶段 vLLM 推理：
 - 阶段 1: 生成 `<think>...</think>` 推理文本。
 - 阶段 2: 在固定 decision prefix 后读取 prompt token 的 logprob。
 - 同时支持单条和 batch。
 - 也保留了旧的“模型自报 logits”路径，但现在不是主流程。

回测模块

- [backtest/backtester.py](#)
 - 端到端批量调度。
 - 先批量跑 Agent 1，再批量跑 Agent 2，然后取真实收益。
- [backtest/dataset_parser.py](#)
 - 支持 `.csv`、`.parquet` 和 Hugging Face 数据集。
 - 统一列名为 `input`、`output`、`answer`、`ticker`。
 - 从 FinGPT 风格 prompt 中抽取新闻正文和日期区间。
- [backtest/price_fetcher.py](#)
 - 用 `yfinance` 拉取日线价格。
 - 计算区间 close-to-close 收益。
 - 带内存缓存和磁盘缓存。
- [backtest/run_backtest.py](#)
 - 命令行入口。

新闻抓取

- [ingestion/news_fetcher.py](#)
 - 支持 `alpaca` 和 `finnhub`。
 - Finnhub 分支有重试、限速和去重逻辑。

核心数据结构

NewsFingerprint

定义在 `agent1/schema.py`，包含：

- 事实抽取字段：
 - `source`
 - `published_at`
 - `headline`
 - `companies_named`
 - `event_keywords`
- 情绪字段：
 - `sentiment_label`
 - `sentiment_score`
 - `sentiment_confidence`

- `sentiment_probabilities`
- `sentiment_logits`
- `calibration_T`
- 透传上下文字段：
 - `article_text`

重要校验：

- `companies_named` 不能为空，否则该新闻会被丢弃。
- `event_keywords` 会统一转成小写。

TradingSignal

定义在 `agent2/schema.py`，包含：

- `ticker`
- `direction: long | short | neutral`
- `strategy_tag: momentum | mean_reversion | event_driven | macro | none`
- `confidence`
- `cot`
- 可选审计字段：
 - `signal_logits`
 - `signal_probabilities`
 - `calibration_T`

推理流程是怎样工作的

Agent 1 的情绪路径

Agent 1 不让模型直接输出最终情绪分数，而是走以下流程：

1. 用 guided decoding 做结构化事实抽取。
2. 用情绪 CoT prompt 生成 `<think>...</think>`。
3. 在 `SENTIMENT_DECISION_PREFIX = "Sentiment: "` 后对三个类别打分：
 - `POSITIVE`
 - `NEGATIVE`
 - `NEUTRAL`
4. 在 Python 侧做后处理：
 - `softmax(logits / CALIBRATION_T)`
 - 取最大概率对应标签
 - 取最大概率作为置信度

如果 logits 解析失败，当前实现不会直接报错，而是退化为均匀概率，并把情绪记作 `NEUTRAL`。

Agent 2 的交易信号路径

Agent 2 也采用相同的两阶段方法，但不是直接打 `BUY/HOLD/SELL`，而是打单字符 token：

- 评分 token：`["A", "B", "C"]`
- 映射关系：

- A -> BUY -> long
- B -> HOLD -> neutral
- C -> SELL -> short

这样做是为了避免直接对 BUY/HOLD/SELL 打分时出现的 tokenizer 和先验偏置问题。

之后 Agent 2 会做 PMI 校正：

- `pmi_logits = raw_logits - null_logprobs`

其中 null-context prior：

- 进程内会缓存。
- 也会持久化到 `PMI_PRIOR_PATH`。
- 当模型路径、decision prefix 或 score tokens 变化时会自动失效重算。

如果 Agent 2 解析失败，会返回 `None`，并落一份诊断文件。

Batch 行为

仓库整体是围绕 batch 设计的。

Agent 1 batch

`extract_fingerprint_batch(article_texts)` 会执行：

1. 一次批量 guided decoding 抽取。
2. 一次批量情绪 CoT 生成。
3. 一次批量情绪 choice 打分。

Agent 2 batch

`generate_signal_batch(fingerprints)` 会执行：

1. 一次批量策略 CoT 生成。
2. 一次批量 A/B/C 打分。

回测 batch size

- `backtest/backtester.py` 中 `_BATCH_SIZE = 10`

每个 batch 的实际执行顺序是：

1. Agent 1 抽取。
2. Agent 1 情绪打分。
3. Agent 2 对有效指纹做策略打分。
4. 逐行拉真实价格并组装结果。

配置与环境变量

主配置在 `config.py` 和 `.env` 中。

重要环境变量

- `FINGPT_MODEL_PATH`
 - 本地模型和 tokenizer 的必要路径。
- `FINGPT_ADAPTER_PATH`
 - 在配置中保留了，但当前代码没有真正接入到 vLLM 加载逻辑。
- `SHARE_SINGLE_LLM_BETWEEN_AGENTS`
 - 两个 agent 共享同一个 vLLM 引擎。
- `FINGPT_CALIBRATION_T`
 - softmax 校准温度。
- `FINGPT_LOGITS_MAX_TOKENS`
 - CoT token 上限。
- `FINGPT_PMI_PRIOR_PATH`
 - Agent 2 的 null prior 磁盘缓存路径。
- `FINGPT_YF_CACHE_PATH`
 - yfinance 收益缓存路径。
- `FINGPT_DIAG_MD_DIR`
 - 调试 markdown 文件输出目录。
- `NEWS_PROVIDER`
 - alpaca 或 finnhub
- `ALPACA_API_KEY`、`ALPACA_API_SECRET`
- `FINNHUB_API_KEY`
- `FINNHUB_TIMEOUT_SEC`
- `FINNHUB_MAX_CALLS_PER_SEC`
- `FINNHUB_MAX_RETRIES`
- `FINNHUB_RETRY_BASE_DELAY_SEC`

一个明显的配置历史遗留

`config.py` 里还有一些 Anthropic 相关常量：

- `ANTHROPIC_API_KEY`
- `CLAUDE_MODEL`
- `CLAUDE_MAX_TOKENS`
- `CLAUDE_THINKING_BUDGET`

但当前真正运行的 Agent 2 实现在 `agent2/reasoner.py`，已经是本地 vLLM 方案，不是 Anthropic 方案。开发时应把这些视为旧设计残留，而不是主流程依赖。

运行时输出

实时流水线输出

- 输出 JSON 到 `output/signals_<timestamp>.json`
- 每条记录是一个序列化后的 `TradingSignal`

回测输出

- 默认输出 CSV 到 `output/backtest_results.csv`
- 每行会包含：
 - 数据集元信息

- Agent 1 情绪结果
- Agent 2 信号结果
- 真实收益
- 仓位
- 策略收益
- `skipped_reason`

常见跳过原因

- `fingerprint_failed`
- `signal_failed`
- `price_fetch_failed`

日志和诊断文件

- Agent 1 markdown 调试输出：
 - `<diag_dir>/agent1`
- Agent 2 markdown 调试输出：
 - `<diag_dir>/agent2`
- Agent 2 失败诊断：
 - `<diag_dir>/agent2_failures`
- Agent 2 CoT 日志：
 - `logs/cot_<ticker>_<timestamp>.txt`

回测指标

`compute_metrics()` 当前会返回：

- `total_rows`
- `successful_rows`
- `skip_rate`
- `direction_accuracy`
- `long_accuracy`
- `short_accuracy`
- `mean_strategy_return`
- `std_strategy_return`
- `annualized_sharpe`
- `total_pnl`
- `vs_fingpt_accuracy`

真实方向标签是通过 `direction_from_return()` 计算的，默认中性阈值为 `0.001`。

依赖情况

依赖列在 `requirements.txt` 中，主要可分为几类：

- 模型与推理：
 - `transformers`
 - `torch`

- `peft`
- 代码实际依赖 `vLLM`，但 `requirements.txt` 里没有列出，部署环境需要手动保证 `vLLM` 可用。
- 数据与评估：
 - `pandas`
 - `pyarrow`
 - `datasets`
 - `yfinance`
- 基础设施：
 - `python-dotenv`
 - `requests`
 - `pydantic`
- 开发：
 - `pytest`
 - `notebook`

开发者常用入口

运行实时流水线

```
python pipeline.py AAPL MSFT NVDA
```

运行完整回测

```
python -m backtest.run_backtest --dataset "FinGPT/fingpt-forecaster-dow30-202305-202405" --metrics
```

小样本 smoke test

```
python -m backtest.run_backtest --dataset "FinGPT/fingpt-forecaster-dow30-202305-202405" --max-rows 50 --metrics
```

指定本地数据和输出路径

```
python -m backtest.run_backtest --dataset "your_data.csv" --output "output/my_backtest.csv" --metrics
```

如果要改代码，应该看哪里

改抽取行为

- Prompt: `agent1/prompt.py`

- 解析与校验: `agent1/extractor.py`、`agent1/schema.py`

改情绪类别或校准方式

- 类别顺序: `config.py`
- 决策前缀和打分 prompt: `agent1/prompt.py`
- softmax 行为: `vllm_logits_client.py` 和 `agent1/extractor.py`

改交易决策逻辑

- 策略 prompt 和 score tokens: `agent2/prompt.py`
- 策略映射和 PMI 逻辑: `agent2/reasoner.py`
- 信号 schema: `agent2/schema.py`

改 batch size 或结果拼装

- `backtest/backtester.py`

改市场数据逻辑

- `backtest/price_fetcher.py`

改新闻源行为

- `ingestion/news_fetcher.py`

当前最需要知道的开发注意点

1. 测试已经和当前实现脱节

`tests/test_extractor.py` 和 `tests/test_reasoner.py` 仍然是旧架构下的测试:

- 还在 `mock_model`、`_tokenizer` 和 Anthropic client。
- 当前实现已经改成 vLLM + prompt logprob。
- 测试里还期待 `figures_quoted` 这类字段, 但当前 `NewsFingerprint` 已没有这些字段。

因此, 当前测试不能作为这个仓库的可靠回归保障。继续开发前, 最好先把测试重写到现有实现上。

2. README 和部分注释带有过渡历史痕迹

仓库中的部分说明文档和注释还保留了旧设计或迁移中的表述。遇到冲突时, 应以以下运行时代码为准:

- `agent1/extractor.py`
- `agent2/reasoner.py`
- `vllm_logits_client.py`
- `backtest/backtester.py`

3. `FINGPT_ADAPTER_PATH` 目前没有真正接入

虽然它出现在配置和 `.env.example` 中, 但当前 `_load_model()` 并没有把它用到 vLLM 加载流程里。

4. vLLM 是硬依赖

虽然代码里是动态导入 vLLM，但运行环境必须真的安装并可用。尤其是在脱离 notebook 或 Colab 复现时，这一点很关键。

建议的开发阅读路径

1. 先看 `config.py`、`.env.example` 和你要用的入口文件。
2. 如果是改推理逻辑，顺着这条链读：
 - prompt 文件
 - schema 文件
 - extractor 或 reasoner
 - `vllm_logits_client.py`
3. 如果是改评估逻辑，顺着这条链读：
 - `dataset_parser.py`
 - `backtester.py`
 - `price_fetcher.py`
4. 先把测试视为待升级项，不要默认它们能保护当前实现。
5. notebook 更适合做实验和补跑，不适合作为理解主架构的唯一来源。